The MoE Memory Wall

Predicting and Prefetching Experts for Faster LLM Inference

Mid-Semester Project Report

Submitted by

Student Name 1 (Roll Number)
Student Name 2 (Roll Number)
Student Name 3 (Roll Number)

Computer Architecture

October 2026

Acknowledgement

We would like to thank the course faculty and teaching assistants for their guidance, and the authors of LLMSimulator and the SSD-offloading framework, which let us run all our experiments through simulation.

Student Name 1
Student Name 2
Student Name 3

Abstract

Mixture-of-Experts (MoE) models like Mixtral-8x7B have 46.7 B parameters but activate only around 13 B per token, because each token is routed to just a couple of experts. The catch is that the unused expert weights are still stored, and during decode every step has to stream most of them out of memory. This is the MoE memory wall, and this project studies it using simulation. We reproduced the Duplex configuration and obtained a 3.10× decode speed-up (23.94 ms to 7.73 ms on 4 GPUs). We then put all weights into a single 80 GiB GPU, found the model does not fit (88.48 GiB needed), and moved the experts to off-chip DRAM, where loading them (68.13 ms) costs 2.72× as much as executing them (25.02 ms). By looking inside one decode step we show why: expert feed-forward networks take 96.2% of decoder time and one expert projection's weight read costs 98× its arithmetic. Finally, we analysed how predictable expert accesses are. All measured numbers come from our own simulation runs, and Appendix A has the commands to reproduce them.

Contents

1 Introduction **1**

2 Background **2**
2.1 Why decode is bandwidth bound 2
2.2 Mixtral-8x7B and why the experts are the problem 2
2.3 The memory hierarchy 2
2.4 Duplex 3
2.5 The simulators 3

3 Experimental Setup **4**

4 Results **5**
4.1 Deliverable 1: reproducing Duplex 5
4.2 Deliverable 2: single GPU with experts in off-chip DRAM 6
4.3 Deliverable 3: where the time goes inside a decode step 7
4.4 Deliverable 4: can expert accesses be predicted? 8

5 Discussion **10**
5.1 What we learned 10
5.2 Limitations 10
5.3 Reproducibility 10

6 Conclusion and Future Work **11**

A Run Commands **13**

Chapter 1

Introduction

Large language models are usually served in two phases. The **prefill** phase processes the whole prompt at once and is mostly limited by compute. The **decode** phase then produces one token at a time, and for every new token the machine has to read the model's weights from memory again while doing only a small amount of arithmetic. So decode is a memory-bandwidth problem: the time for one step is roughly the bytes of weights divided by the memory bandwidth, no matter how tiny the math is.

Mixture-of-Experts (MoE) makes this harder. Instead of one big feed-forward network per layer, an MoE layer has many "experts" (8 in Mixtral) and a small router that picks the top-2 for each token. Model capacity grows, but the weights of all the unused experts still sit in memory, and in decode nearly all of them get touched every step because the batch is small. That is the problem this project is about: **the MoE memory wall**.

For this project we had to:

1. Reproduce the Duplex configuration in LLMSimulator and show a working simulation (D1).
2. Disable multi-GPU sharding and Logic-PIM, put all expert weights in off-chip DRAM, and measure expert loading time versus expert execution time (D2).
3. Understand the MoE execution flow and explain why expert loading is the bottleneck (D3).
4. Check whether expert accesses are predictable, using plots and statistics, as the first step towards a prefetcher (D4).

Chapter 2 gives the background, Chapter 3 our setup, Chapter 4 the results for each of the four items above, Chapter 5 what we learned and what is still open, and Chapter 6 the conclusion and future work.

Chapter 2

Background

2.1 Why decode is bandwidth bound

For a weight matrix of $k \times n$ elements, one decode step with m tokens reads $2kn$ bytes from memory and performs $2mkn$ floating-point operations. The arithmetic intensity (operations per byte) is therefore just m, the number of tokens. A kernel's runtime is bounded by the roofline model of Williams, Waterman and Patterson [5]: its time is

$$t = \max\left(\frac{\text{FLOPs}}{\text{peak FLOPs}},\ \frac{\text{bytes}}{\text{bandwidth}}\right), \tag{2.1}$$

and below the ridge point (peak FLOPs divided by bandwidth) a kernel is memory bound. For an FP16 H100 that ridge point is 989.4 TFLOPS/3.352 TB/s $\approx$ 295 Op/B. In MoE decode an expert sees only $m \approx$ 1–3 tokens, roughly 100–300$\times$ below the ridge (295/3 for the m=3 op analysed in D3), so expert projections spend almost all their time reading weights.

2.2 Mixtral-8x7B and why the experts are the problem

Mixtral [4] has 32 layers, hidden size 4096, grouped-query attention, and 8 experts per layer routed top-2. Each expert is a SwiGLU feed-forward network with three matrices, so

$$\text{one expert} = 3 \times 4096 \times 14336 = 176.16\text{M parameters} = 352.3\text{ MB in FP16}. \tag{2.2}$$

The experts therefore make up 45.10 B of the model's 46.7 B parameters — the model's weights alone occupy 86.99 GiB in FP16. Each of the 8 experts per layer is used by only a couple of tokens per step, but at batch 8 a mean of 7.28 of 8 experts is active per layer, so nearly the full 2.82 GB of expert weights per layer is streamed every step.

2.3 The memory hierarchy

The GPU side is H100 HBM3 [6]: 80 GiB capacity, 3.352 TB/s bandwidth. When experts are placed off-chip they live in CPU DRAM behind an NVLink-C2C 5.0 link of 900 GB/s — 3.7$\times$ slower to reach. (A host CPU behind PCIe would be slower still, so the 68.13 ms loading time in D2 is optimistic.) Throughout this report, footprints are quoted in GiB because that is how the simulator prints them, while traffic is in GB because link speeds are quoted in GB/s.

2.4 Duplex

Duplex attaches a Logic-PIM die to the *same* HBM stacks as the GPU [1]. The GPU handles compute-heavy work while the Logic-PIM serves the bandwidth-heavy small-batch decode experts, and since both sit on the same HBM no extra data movement is needed. In LLMSimulator this is the `GPU+LOGIC` processor type.

2.5 The simulators

We used two analytical simulators (they evaluate the roofline equations directly, so re-runs are bit-identical):

- **LLMSimulator** [7] builds the model's operation graph and reports, for every operator, a fused time — the maximum of compute time and memory time — and an energy, all from a YAML config. Its routing is synthetic: uniform by default, or Zipf-skewed if `skewness` is set.
- **SSD-offloading** [8] builds on it with an offload model: experts can live in CPU DRAM and `expert_ffn` measures either the CPU→HBM copy time (`collects_gpu_compute: off`) or the GPU's execution time (`on`). It only reads `./config.yaml` from the working directory, which cost us some confusion early on.

Chapter 3

Experimental Setup

All runs use the Mixtral model in FP16 on an H100-class device. The configs we wrote for this report (Table 3.1; the last two rows are the D1 isolation variants, i.e. the D1 configs with one field flipped each):

Table 3.1: Simulation configurations (all Mixtral, FP16).

Config	Simulator	Setup	Used for
`mixtral_gpu.yaml`	LLMSim	4 GPUs, DP4, batch 32	D1 baseline
`mixtral_duplex.yaml`	LLMSim	4 GPUs, `GPU+LOGIC`, TP4	D1 Duplex
`mixtral_single_gpu.yaml`	LLMSim	1 GPU, batch 8, op log on	D2 OOM, D3 flow
`cpu_copy.yaml`	SSD-off	experts in CPU DRAM, copy mode	D2 loading
`cpu.yaml`	SSD-off	experts in CPU DRAM, compute mode	D2 execution
GPU config + TP4	LLMSim	both tensor degrees 4	D1 isolation (TP4)
Duplex config, exec off	LLMSim	`parallel_execution` off	D1 isolation (PIM)

The two isolation runs flip exactly one field of a D1 config each: the GPU config gets both tensor degrees set to 4, and the Duplex config gets `parallel_execution` off. Their logs and CSVs are kept in `rerun_backup/`. The two SSD configs live in `ssd_cfgs/` and differ only in the `collects_gpu_compute` flag.

Both simulators were built in an Ubuntu 22.04 container (with `g++-11`, C++17; `git` is needed at configure time). Each LLMSimulator run writes a log with the memory footprint and a total time, plus one CSV file whose rows are the iterations; each SSD-offloading run likewise writes one CSV with per-operation latencies and energies. Three short Python scripts (`analysis_d2.py`, `analysis_d3.py`, `analysis_d4.py`) parse these outputs and re-derive the numbers; exact commands are in Appendix A. One quirk to know: `iter: 3` only writes two CSV rows, because the export happens *before* each iteration, so the last one never gets written — this is upstream simulator behaviour [7], not a change of ours. We always used row 0. The two rows agree to 0.4% on total latency and to 0.6% on the per-op columns; the single-GPU run's two rows differ by 6%.

Chapter 4

Results

4.1 Deliverable 1: reproducing Duplex

We ran the same workload on 4 H100s, once as a plain GPU system and once as Duplex (Table 4.1). Both are decode-only slices: batch 32, 1024 input / 128 output tokens, FP16.

Table 4.1: D1 results: plain GPU versus Duplex. Ratios are computed from the exact CSV values, so they can differ in the last digit from the rounded μs shown.

Metric	GPU	Duplex	Change
Total time, 3 iterations (log)	23.943 ms	7.726 ms	**3.10**×
Latency per iteration (CSV)	7.980 ms	2.574 ms	3.10×
Expert FFN (`expert_ffn`)	6765 μs	1721 μs	3.93×
Attention (`atten_gen`)	337 μs	81 μs	4.14×
QKV / output projections	482 / 322 μs	124 / 83 μs	∼3.9×
Communication	69 μs	545 μs	TP all-reduce
Energy per iteration	3.228 J	3.160 J	−2.1%

Duplex is **3.10× faster** at decode. Most of the win is `expert_ffn`, which is 85% of a step's time (6765 of 7980 μs): the Logic-PIM serves the experts straight from the shared HBM. The projection weights shrink $\sim 4\times$ from tensor-parallel sharding, and the new 545 μs all-reduce is the price of that sharding; `atten_gen` (337 to 81 μs) is cut by the Logic-PIM, since TP4 alone leaves it unchanged. The benefit is latency, not energy — energy only improves by 2.1%.

To see which of the three changed fields is responsible, we ran the two isolation variants of Table 3.1. TP4 sharding *alone* (without Logic-PIM) barely changes anything (23.943 to 23.641 ms, 1.3% = a factor of 1.013), because the experts are bandwidth bound and sharding their weights does not reduce the bytes each device reads. Turning the Logic-PIM on gives the real jump (23.641 to 8.258 ms, 2.86×), and `parallel_execution` on adds another 7%. Chained, $1.013 \times 2.86 \times 1.07 = 3.10\times$, matching the direct measurement. So the speed-up is overwhelmingly Logic-PIM: 2.86× of the 3.10×.

Two honest notes: this slice is decode-only, which is the *best* case for Logic-PIM (the paper's $\sim 2.5\times$ is a broader average), and the printed footprints — 26.07 GiB for the GPU run and 23.96 GiB for Duplex — differ because under data parallelism the experts are split 4 ways while attention is replicated, whereas under TP4 all shardable weights are split and embeddings stay replicated.

4.2 Deliverable 2: single GPU with experts in off-chip DRAM

The model does not fit. With all weights in one H100, the log reads ACT: 0.95 GiB, Weight: 86.99 GiB, Cache: 0.53 GiB, **Total: 88.48 GiB** — more than the 80 GiB of HBM (the CSV even raises an OOM flag). Since the weights alone exceed the HBM, no batch size would fix this. Hence the project brief's plan: experts go to off-chip CPU DRAM, reached over the 900 GB/s link.

We then ran the SSD-offloading framework with experts in CPU memory (batch 4, one decode step over all 32 layers). The same `expert_ffn` number means two different things depending on one flag (Table 4.2):

Table 4.2: D2: the `expert_ffn` column measures loading or execution depending on one flag.

Config	`collects_gpu_compute`	`expert_ffn` means	Result
`ssd_cfgs/cpu_copy.yaml`	off	expert copy CPU→HBM	**68.13 ms**
`ssd_cfgs/cpu.yaml`	on	GPU expert execution	**25.02 ms**

So **loading the experts costs** 2.72× **as much as executing them.** The numbers also match theory to within 0.1%. The framework's offload energy implies the run moves 61.31 GB of expert bytes (cross-check: 68.13 ms × 900 GB/s = 61.32 GB): that is 174 expert loads of 352.3 MB, i.e. 5.44 distinct experts per layer, which is what you would expect from 4 tokens drawing 2 of 8 experts each:

$$8\left(1-\left(\tfrac{6}{8}\right)^4\right) = 5.47. \tag{4.1}$$

Then

$$\text{load} = \frac{\text{bytes}}{900\ \text{GB/s}}, \qquad \text{exec} = \frac{\text{bytes}}{3352-900\ \text{GB/s}}, \qquad \frac{\text{load}}{\text{exec}} = \frac{2452}{900} = 2.72\times. \tag{4.2}$$

The striking part: "execution" is itself memory bound. Its arithmetic is only 0.091 ms — 90.2 GFLOP at 989.4 TFLOPS — which is 274× less than the measured 25.02 ms; execution is pure HBM bandwidth. (The FLOP count charges each of the 256 routed slots, while the copy only moves the 174 distinct experts.)

So the load/exec ratio is a *leftover-bandwidth* identity, not plain link-versus-HBM (which would be 3352/900 = 3.72×). While the link copies the experts into HBM at 900 GB/s, that copy traffic itself consumes 900 GB/s of HBM's bandwidth, so execution is left with only 3352 − 900 = 2452 GB/s. The framework models exactly this: it sets `memory_bandwidth` to `base_memory_bandwidth - offload_bandwidth` (`hardware_config.h:97`), divides the expert execution time by that reduced bandwidth, and divides the copy time by the raw link bandwidth (`linear_impl.cpp:55, 62`). Hence

$$\frac{\text{load}}{\text{exec}} = \frac{\text{HBM}-\text{link}}{\text{link}} = \frac{2452}{900} = 2.72\times. \tag{4.3}$$

In energy terms the CPU→HBM copy costs 3.19× the HBM read (6.57 J vs 2.06 J), which follows from the framework's per-bit constants (13.4 vs 4.2 pJ/bit).

Two things to keep in mind when reading these numbers. The framework has no expert caching — it reloads every routed expert every step, which is the pessimistic on-demand baseline, not a bug. And in copy mode the offload flag applies to *every* linear layer, not just the experts, so the attention projections are charged a copy even though their weights never leave the GPU:

the copy-mode CSV shows `qkvgen` = 1.79 ms and `o_proj` = 1.19 ms, which is exactly those weights' bytes across all 32 layers (1.61 GB and 1.07 GB) divided by the 900 GB/s link.

Hence only the expert-only columns mean what they say: quoting the copy-mode layer total (`MoE_decoders`, 71.23 ms) would overstate the loading time, so we use `expert_ffn` (68.13 ms) and nothing else.

4.3 Deliverable 3: where the time goes inside a decode step

We re-ran the single-GPU config with the operation log on (this changes no results). Layer 0 of the parsed op tree below — `expertFFN` in the log is the same quantity as `expert_ffn` in the CSV:

```
MoE_decoder_0            872.5us
  attention               30.5us       <- 3.5% of the layer
    attn_qkv_proj         15.1us   [8x4096] x [4096x6144]
    self_attention         5.4us   (flash attention)
    attn_o_proj           10.0us
  expertFFN              841.8us       <- 96.5% of the layer
    gate_fn               0.039us   4096 -> 8 router scores
    moe_route             0us       tokens/expert = [3, 2, 2, 1, 2, 2, 3, 1]
    expert_FFN_0          105.3us
      gate_proj           35.1us   [3x4096] x [4096x14336]
      up_proj             35.1us   [3x4096] x [4096x14336]
      down_proj           35.1us   [3x14336] x [14336x4096]
    expert_FFN_1..7       ~105us each, back-to-back
```

The simulator reports one fused time per op, max(compute, memory), so we can check what dominates by replaying the arithmetic for layer 0's first active `gate_proj` ($[3 \times 4096] \times [4096 \times 14336]$):

$$\begin{aligned} \text{weight read} &= 4096 \times 14336 \times 2\,\text{B} = 117.4\ \text{MB}, \\ t_{\text{mem}} &= 117.4\ \text{MB}/3.352\ \text{TB/s} = 35.036\ \mu\text{s} \quad (\text{measured } 35.069\ \mu\text{s}), \\ t_{\text{cmp}} &= 352.3\ \text{MFLOP}/989.4\ \text{TFLOPS} = 0.356\ \mu\text{s}. \end{aligned} \tag{4.4}$$

Reading the weights takes **98**× longer than the math, and with arithmetic intensity of 3 Op/B against the H100 ridge of 295 Op/B this op is deeply memory bound. The factor is simply the ridge point divided by the number of tokens an op serves: $295/3 \approx 98\times$ here. The expert `up_proj` and `down_proj` have the same 117.4 MB weight shape, so the same holds for them; the attention projections serve all 8 tokens, so their factor is only $295/8 \approx 37\times$ — memory bound too, but smaller, which is why the expert FFN block dominates. Over all 32 layers, expert FFNs are **96.2%** of decoder time and a mean of **7.28 of 8** experts is active per layer.

So loading is the bottleneck simply because the bytes are many and the tokens are few: a 352.3 MB expert serves 1–3 tokens, and once experts live off-chip those bytes cross a 3.7× narrower pipe. The measured 2.72× of D2 is a different quantity: it is not the pipe ratio (3352/900) but the leftover-bandwidth ratio (2452/900), because the copy consumes 900 GB/s of HBM while the GPU executes.

4.4 Deliverable 4: can expert accesses be predicted?

LLMSimulator ships no real routing traces, so `analysis_d4.py` generates three scenarios itself: the simulator's uniform routing, its Zipf-skewed routing ($s = 1.0$), and one *synthetic* ST-MoE-like trace with stickiness (a token repeats the previous token's experts with probability 0.7, and otherwise each expert from the previous layer is kept with probability 0.6). The third one is not a measurement of real Mixtral routing — its correlation is set by those two knobs — but it lets us test the analysis method (Table 4.3).

Table 4.3: D4: routing statistics from `analysis_d4.py`. The uniform row's coverage is the script's re-implementation of default routing; the simulator's own batch-8 run gives 7.28 (D3).

Routing	Top expert share	Layer overlap	Token overlap	Experts/step
uniform (modelled default)	0.126	0.251	0.249	7.24
Zipf s=1.0 (skewness mode)	0.315	0.362	0.361	6.45
ST-MoE-like (synthetic)	0.286	0.556	0.825	6.56

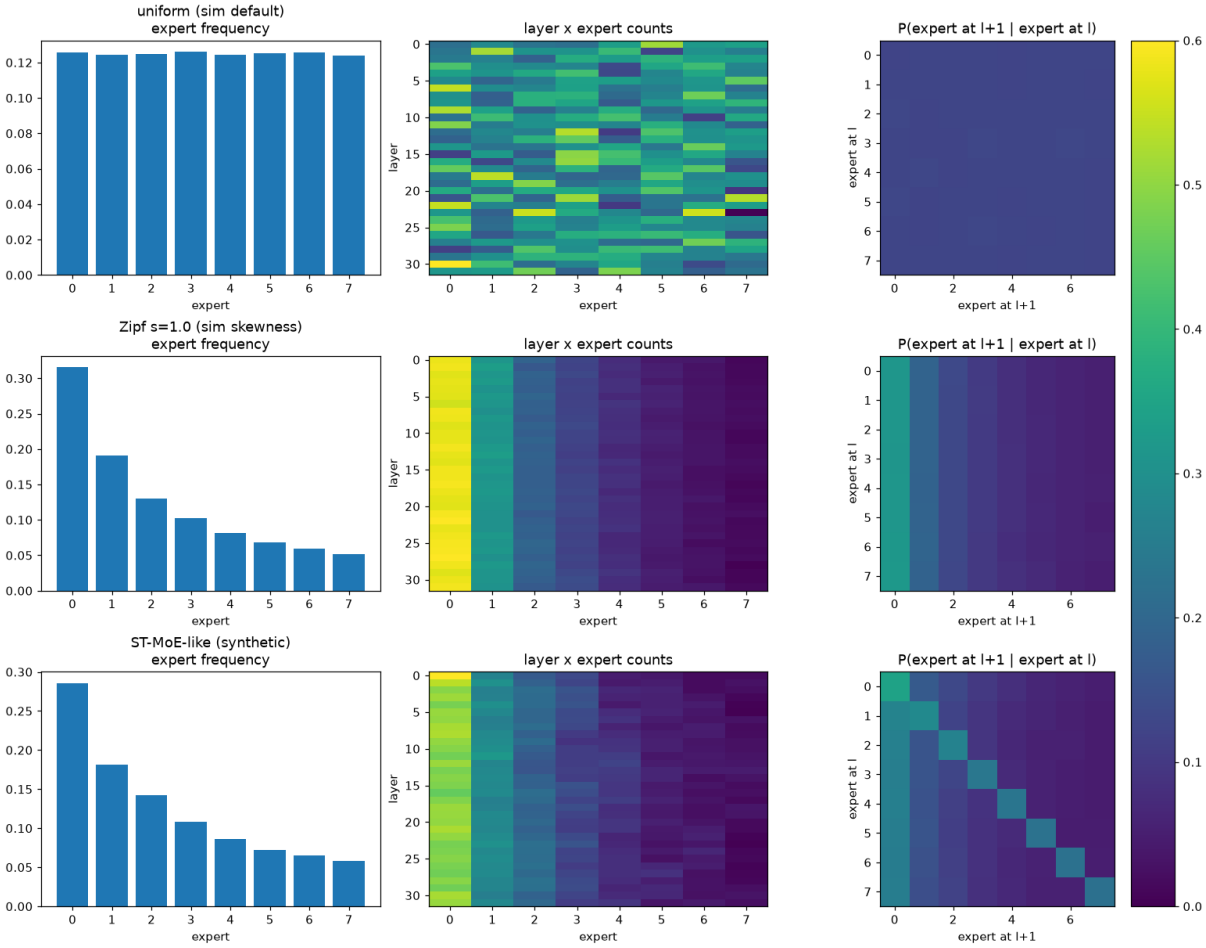


Figure 4.1: Generated by `analysis_d4.py`: expert frequency, layer-by-expert counts and the transition matrix for each routing scenario. Only the correlated trace has a diagonal (predictable) transition matrix.

The overlap expected by pure chance is $2/8 = 0.25$; Figure 4.1 shows the same statistics as plots. Three things stand out:

- The simulator's routing has **no** correlation — both overlaps sit on the 0.25 null and the transition matrix is flat. Any predictor we build must be evaluated on traces with real structure.
- Zipf skew only concentrates traffic on hot experts (one expert takes 31.5% of accesses, overlap rises to 0.36). The transition matrix shows vertical stripes — *popularity*, not *predictability*.
- Only the sticky trace gives a diagonal transition matrix and overlaps well above the null (0.556 layer-to-layer, 0.825 token-to-token) — that diagonal is what prediction exploits.

The consequence for our prefetcher plans: correlation can tell us *which* experts to fetch early, but it barely reduces *how many* we must fetch — coverage stays between 6.45 and 7.24 of 8 experts in every case.

Chapter 5

Discussion

5.1 What we learned

The four results tell one story. Decode is bandwidth bound; expert weights are huge (352.3 MB each) and see only 1–3 tokens, so 96.2% of decoder time is spent reading expert weights (D3). Moving those experts off-chip puts them behind a 900 GB/s link, and because the copy and the GPU's execution contend for HBM, execution is left with only 2452 GB/s — which is what makes loading cost 2.72× what execution costs (D2). A PIM die that shares the HBM removes that extra hop for the bandwidth-bound experts, and that is where the 3.10× comes from (D1). Whether prefetching can hide the rest depends on routing correlation (D4), which the stock simulator does not have.

5.2 Limitations

- Both simulators are analytical: no DRAM timing, no contention, no queueing. The numbers are idealized bounds, not measurements from real hardware.
- The D2 framework has no expert caching, so 68.13 ms is the pessimistic on-demand cost.
- The D1 speed-up is a decode-only slice, the best case for Logic-PIM; a mixed prefill+decode workload would show less.
- D4's correlated trace is synthetic. It validates our analysis, not real Mixtral routing — real traces are needed before claiming any prediction accuracy.
- The 900 GB/s off-chip link is a fast interconnect; a slower host would make loading dominate even more.

5.3 Reproducibility

Everything was regenerated from the files in the project folder: both simulators were rebuilt from scratch in a clean container, D1 and single-GPU logs came out byte-identical, the D2 columns reproduced exactly (one unused energy column drifted by 5×10^{-8}, a floating-point artifact), and all three analysis scripts exit 0. Build and run commands are in Appendix A.

Chapter 6

Conclusion and Future Work

We reproduced Duplex and measured a 3.10× decode speed-up, almost all of it from the Logic-PIM. We showed with a memory accounting that Mixtral-8x7B cannot fit in one 80 GiB GPU (88.48 > 80 GiB), so its experts must live in off-chip DRAM, and measured that loading the experts costs 2.72× as much as executing them — a ratio that reduces exactly to the leftover-bandwidth identity (HBM − link)/link = 2452/900, because the copy consumes 900 GB/s of HBM while the GPU executes. From the operation tree we established where the decoder step spends its time: expert FFNs are 96.2% of decoder time and one expert projection's weight read costs 98× its arithmetic. Finally, we showed that the stock simulator's uncorrelated routing leaves nothing to predict, and that structure in the trace is what a prefetcher needs — and that even then correlation changes *which* experts to fetch, not *how many*.

For the end-semester phase we plan to:

1. Implement a training-free expert predictor in the spirit of ST-MoE [2] and APEX [3], and score it against the experts actually routed.
2. Build a prefetcher that fetches predicted experts into HBM ahead of use, overlapping loading with attention.
3. Compare with on-demand execution on latency, exposed stalls, extra traffic and energy.

Bibliography

[1] S. Yun et al., "Duplex: A device for large language models with mixture of experts, grouped query attention, and continuous batching," arXiv:2409.01141 [cs.AR], 2024.

[2] Y. Zhao, R. Bunescu, A. Louri, A. Karanth, and K. Wang, "A spatio-temporal expert prefetching framework for efficient MoE-based LLM inference," arXiv:2606.15453 [cs.AR], 2026.

[3] A. Kanani, L. Badawi, and U. Y. Ogras, "APEX: Adaptive expert prefetching for memory-efficient edge MoE inference," arXiv:2608.11688, 2026.

[4] A. Q. Jiang et al., "Mixtral of experts," arXiv:2401.04088, 2024.

[5] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 60–67, 2009.

[6] NVIDIA Corporation, "NVIDIA H100 Tensor Core GPU architecture," whitepaper, 2022.

[7] scale-snu, "LLMSimulator," `https://github.com/scale-snu/LLMSimulator`.

[8] scale-snu, "SSD-offloading," `https://github.com/scale-snu/SSD-offloading`.

Appendix A

Run Commands

```
cd ~/Downloads/CA_Project        # the project folder

# build (Ubuntu 22.04 container with g++-11, cmake, make, git)
cd LLMSimulator && mkdir -p build data && cd build && \
  cmake -DCMAKE_CXX_COMPILER=g++-11 -DCMAKE_C_COMPILER=gcc-11 -
      DCMAKE_CXX_STANDARD=17 .. && \
  make -j4
cd ../../SSD-offloading/LLMSimulator && mkdir -p build && cd build && \
  cmake -DCMAKE_CXX_COMPILER=g++-11 -DCMAKE_C_COMPILER=gcc-11 -
      DCMAKE_CXX_STANDARD=17 .. && \
  make -j4 run
cd ../../..                      # back to the project folder

# D1 + D2 OOM + D3 op tree
cd LLMSimulator/build && \
  ./run ../../mixtral_gpu.yaml        > ../../run_mixtral_gpu.log && \
  ./run ../../mixtral_duplex.yaml     > ../../run_mixtral_duplex.log && \
  ./run ../../mixtral_single_gpu.yaml > ../../run_mixtral_single.log

# D1 isolation: the same D1 configs with one field flipped each
#   mixtral_gpu.yaml with expert_tensor_degree/none_expert_tensor_degree = 4
#   mixtral_duplex.yaml with parallel_execution = off
# their logs and CSVs are kept in rerun_backup/

# D2 load vs exec (the SSD framework reads ./config.yaml from the CWD)
cd ../../SSD-offloading/LLMSimulator/build && \
  cp ../../../ssd_cfgs/cpu.yaml config.yaml      && ./run > ../../../
      ssd_results/ssd_cpu.log && \
  cp ../../../ssd_cfgs/cpu_copy.yaml config.yaml && ./run > ../../../
      ssd_results/ssd_cpu_copy.log && \
  cp tmp/*.csv ../../../ssd_results/
cd ../../..                      # back to the project folder

# analysis
python3 analysis_d2.py
python3 analysis_d3.py
python3 analysis_d4.py     # needs numpy + matplotlib, writes d4_correlation
    .png
```